

Smytten E-Commerce Database Management System

CHAPTER – 4

ANALYZING FUNCTIONAL DEPENDENCIES, REDUNDANCY AND DATA ANOMALIES

4.1 INTRODUCTION

The Smytten E-Commerce Database Management System is designed to efficiently manage customers, administrators, brands, categories, products, carts, wishlists, orders, payments, deliveries, reviews, and inventory. Functional dependencies help identify the relationships between attributes in each table, while redundancy analysis and data anomaly detection help maintain data consistency and reduce unnecessary duplication. By organizing data into properly structured and related tables, the database minimizes redundancy and prevents insertion, updation, and deletion anomalies. A well-designed database improves data integrity, ensures efficient data retrieval, and supports the smooth operation of the Smytten e-commerce platform.

1. CUSTOMER TABLE

Functional Dependency

Customer_ID → Customer_Name, Email, Phone_Number, Address

Redundancy Analysis:

Customer information is stored only once in the Customer table. Other tables such as Orders, Cart, Wishlist, and Review store only the Customer_ID as a foreign key instead of repeating customer details.

Data Anomalies:

- **Insert Anomaly:** A new customer can be added to the database without creating an order, cart, wishlist, or review.
- **Update Anomaly:** Updating a customer's phone number or address requires changes only in the Customer table.
- **Delete Anomaly:** Deleting an order or review does not remove the customer's information from the database.

2. ADMIN TABLE

Functional Dependency

Admin_ID → Admin_Name, Email, Password

Redundancy Analysis:

Administrator details are stored separately in the Admin table. This avoids repeating administrator login information across other tables.

Data Anomalies:

- **Insert Anomaly:** A new administrator can be added without creating any product or order records.
- **Update Anomaly:** Updating an administrator's email or password requires modification only in the admin table.
- **Delete Anomaly:** Deleting an administrator does not affect customer, product, or order information.

3. BRAND TABLE

Functional Dependency

Brand_ID → Brand_Name, Brand_Type

Redundancy Analysis:

Brand information is stored only once in the Brand table. Products reference the brand using Brand_ID instead of repeatedly storing the brand name and type.

Data Anomalies:

- **Insert Anomaly:** A new brand can be added even before any product is associated with it.
- **Update Anomaly:** Changing a brand name or brand type requires updating only one record.
- **Delete Anomaly:** Deleting a product does not remove the corresponding brand information.

4. CATEGORY TABLE

Functional Dependency

Category_ID → Category_Name

Redundancy Analysis:

Each category is stored only once in the Category table. Products reference the category using Category_ID, preventing repeated storage of category names.

Data Anomalies:

- **Insert Anomaly:** A new category can be added without adding any products.
- **Update Anomaly:** Renaming a category requires modification in only one place.
- **Delete Anomaly:** Deleting a product does not delete the category information.

5. PRODUCT TABLE

Functional Dependency

Product_ID → Product_Name, Price, Stock, Brand_ID, Category_ID

Redundancy Analysis:

Product details are stored only in the Product table. Other tables such as Cart, Wishlist, Review, and Inventory use Product_ID to reference the product instead of repeating product information.

Data Anomalies:

- **Insert Anomaly:** A new product can be added before it is added to a cart, wishlist, or order.
- **Update Anomaly:** Updating the product price or stock requires changes only in the Product table.
- **Delete Anomaly:** Deleting a cart or wishlist record does not remove the product information.

6. CART TABLE

Functional Dependency

Cart_ID → Customer_ID, Product_ID, Quantity

Redundancy Analysis:

The Cart table stores only cart-related information. Customer and product details are referenced using Customer_ID and Product_ID instead of repeating their complete information.

Data Anomalies:

- **Insert Anomaly:** A product can be added to a customer's cart without creating an order.
- **Update Anomaly:** Changing the quantity affects only the selected cart record.
- **Delete Anomaly:** Removing a product from the cart does not delete the customer or product information.

7. WISHLIST TABLE

Functional Dependency

Wishlist_ID → Customer_ID, Product_ID

Redundancy Analysis:

The Wishlist table stores the relationship between customers and their selected products. It uses Customer_ID and Product_ID to reference existing records instead of repeating customer and product details.

Data Anomalies:

- **Insert Anomaly:** A product can be added to a customer's wishlist without placing an order.
- **Update Anomaly:** Changes to customer or product information are handled in their respective tables.
- **Delete Anomaly:** Removing a wishlist item does not delete the customer or product record.

8. ORDERS TABLE

Functional Dependency

Order_ID → Customer_ID, Order_Date, Total_Amount, Order_Status

Redundancy Analysis:

The Orders table stores only order-related information. Customer details are referenced using Customer_ID, while payment and delivery information are maintained separately in their respective tables.

Data Anomalies:

- **Insert Anomaly:** A new order can be created using an existing Customer_ID without duplicating customer information.
- **Update Anomaly:** Updating the order status requires modification only in the Orders table.
- **Delete Anomaly:** Deleting an order does not remove the customer's information from the Customer table.

9. PAYMENT TABLE

Functional Dependency

Payment_ID → Order_ID, Payment_Mode, Payment_Status

Redundancy Analysis:

Payment information is stored separately from the Orders table. The Order_ID is used to link payment information with the corresponding order, preventing repeated payment details.

Data Anomalies:

- **Insert Anomaly:** Payment details can be recorded for an existing order without duplicating order information.
- **Update Anomaly:** Updating the payment status requires modification only in the Payment table.
- **Delete Anomaly:** Deleting a payment record does not remove the corresponding order.

10. DELIVERY TABLE

Functional Dependency

Delivery_ID → Order_ID, Delivery_Partner, Delivery_Status

Redundancy Analysis:

Delivery information is maintained separately in the Delivery table. The Order_ID connects delivery details with the corresponding order, avoiding repetition of order information.

Data Anomalies:

- **Insert Anomaly:** A delivery record can be created after an order has been confirmed.
- **Update Anomaly:** Updating the delivery status requires changes only in the Delivery table.
- **Delete Anomaly:** Deleting delivery information does not remove the order details.

11. REVIEW TABLE

Functional Dependency

Review_ID → Customer_ID, Product_ID, Rating, Review_Text

Redundancy Analysis:

The Review table stores customer feedback and ratings separately. Customer_ID and Product_ID are used as foreign keys to reference the respective customer and product records.

Data Anomalies:

- **Insert Anomaly:** A review can be added for a product by a customer without duplicating customer or product information.
- **Update Anomaly:** Updating a rating or review text affects only the selected review record.

- **Delete Anomaly:** Deleting a review does not remove the associated customer or product information.

12. INVENTORY TABLE

Functional Dependency

Inventory_ID → Product_ID, Available_Stock, Last_Updated

Redundancy Analysis:

Inventory information is maintained separately from the Product table to manage stock-related details efficiently. The Product_ID connects inventory information with the corresponding product.

Data Anomalies:

- **Insert Anomaly:** An inventory record can be created for a product without duplicating complete product information.
- **Update Anomaly:** Updating available stock or the last updated date requires modification only in the Inventory table.
- **Delete Anomaly:** Deleting an inventory record does not remove the corresponding product information.

CONCLUSION:

The Smytten E-Commerce Database Management System is designed using appropriate functional dependencies and well-structured relational tables. Each entity stores only the data relevant to its purpose, while primary keys uniquely identify records and foreign keys establish relationships between related tables. The analysis of redundancy shows that data duplication is minimized by storing information only once and referencing it through foreign keys. The database design also helps prevent insertion, updation, and deletion anomalies. Overall, this structured approach improves data consistency, maintains data integrity, reduces redundancy, and supports efficient database management for the Smytten e-commerce platform.